Tarea 1: Plataforma de análisis de preguntas y respuestas en Internet

Profesor: Nicolás Hidalgo
Ayudantes: Isidora González, Natalia Ortega, Joaquín Villegas, Vicente Díaz y Benjamín Aceituno

Introducción y Contexto

En el contexto del crecimiento urbano y la disponibilidad de datos geoespaciales a gran escala, diversas organizaciones de logística urbana (como servicios de reparto, distribución de paquetes y transporte de insumos) requieren acceder y procesar información sobre la distribución de edificaciones en distintas zonas. Para ello se utilizará el dataset **Google Open Buildings** [1], que contiene información sobre la ubicación, tamaño y nivel de confianza de edificaciones detectadas mediante imágenes satelitales.
La zona de estudio corresponde a la Región Metropolitana de Santiago de Chile y estas consultas permiten analizar la densidad de construcciones, comparar regiones y evaluar la calidad de los datos, todo esto buscando optimizar rutas, demanda, y en general mejorar la eficiencia del servicio.
Sin embargo, debido al gran volumen de información y a la alta concurrencia de organizaciones y sistemas que suelen consultar repetidamente las mismas zonas de interés (por ejemplo, centros urbanos o zonas comerciales), el procesamiento de estas consultas puede generar altos tiempos de respuesta y un uso intensivo de recursos. En este contexto, se abre la oportunidad de optimizar las consultas a los datos a través del diseño de un sistema distribuido que incorpore mecanismos de caché para optimizar el acceso a la información, reducir la latencia de respuesta y mejorar el rendimiento del sistema bajo distintos patrones de carga.

1. Metodología

El desarrollo del proyecto se estructurará de manera modular a lo largo del semestre, articulándose en tres entregas principales. Cada una de estas entregas se centrará en un objetivo específico, aunque no serán mutuamente excluyentes, se espera que los módulos o servicios desarrollados en fases anteriores puedan ser reutilizados y adaptados según las necesidades de las entregas posteriores.

La evaluación de cada etapa se fundamentará en la presentación de un **informe técnico** exhaustivo. Dicho documento deberá detallar el proceso de diseño e implementación, incluyendo las métricas de rendimiento, la comparación de resultados, un análisis crítico de la solución propuesta y una sección de preguntas y respuestas. Es un requisito fundamental que todas las decisiones de diseño y arquitectura estén debidamente justificadas y que el comportamiento del sistema esté respaldado por datos empíricos obtenidos durante la fase de evaluación. Adicionalmente, se deberá entregar un **video de demostración** que muestre el funcionamiento de los servicios implementados en cada hito.

Las tres entregas programadas abordan los hitos clave del ciclo de vida del proyecto, definidos de la siguiente manera:

1. **Datos y arquitectura base:** Abocados en la recuperación, gestión y almacenamiento eficiente de los datos y eventos que alimentan el sistema.

2. **Procesamiento y Fallback:** Centrado en la preparación y transformación de los datos para su posterior análisis. Esta etapa incluye la implementación de colas de procesamiento y mecanismos de *fallback* para gestionar solicitudes que presenten errores.

3. **Visualización:** Abocado a la creación de una interfaz o *dashboard* que provea una vista agregada y comprensible de las métricas más relevantes del sistema.

Para garantizar la portabilidad y reproducibilidad del entorno de desarrollo, se utilizará **Docker** [2] como tecnología de virtualización de recursos para el despliegue de los distintos módulos. No se impondrá un *stack* tecnológico

[1] `https://sites.research.google/gr/open-buildings/`
[2] https://www.docker.com/

específico. El equipo de desarrollo tendrá la libertad de seleccionar las herramientas, lenguajes y metodologías que considere más apropiados, siempre y cuando cada elección esté justificada en el informe correspondiente.

2. Descripción del Sistema

La arquitectura del sistema deberá estar estructurada en módulos independientes y cohesivos, facilitando su mantenimiento, escalabilidad y la futura integración con componentes externos. Se definen cuatro servicios principales que se comunicarán de manera secuencial:

1. *Generador de Tráfico:* Simula solicitudes de empresas de reparto y logística que consultan zonas de Santiago. Genera consultas automáticamente siguiendo dos distribuciones de tasa de arribo, las cuales son la Ley de potencia (Zipf) y distribución uniforme respectivamente. Cada consulta incluye el tipo de operación (Q1–Q5), la zona geográfica (bounding box) y los parámetros asociados. Las consultas son completamente sintéticas: se construyen a partir de las zonas predefinidas y los parámetros del dataset, sin interacción con una base de datos externa.

2. *Caché:* Intercepta todas las consultas entrantes. Si la respuesta ya está almacenada (*cache hit*), la retorna directamente y registra el evento en el sistema de métricas. Si no (*cache miss*), delega al Generador de Respuestas. Implementado sobre **Redis** con soporte nativo para TTL y políticas de evicción configurables (LRU, LFU, FIFO).

3. *Generador de Respuestas:* Procesa las consultas delegadas por el caché. Computa el resultado directamente en memoria a partir de los datos del dataset precargados, simulando el tiempo de procesamiento real de una consulta geoespacial. Devuelve la respuesta al caché para su almacenamiento y al sistema de métricas.

4. *Almacenamiento de Métricas:* Registra todos los eventos del sistema: hits, misses, latencias, throughput y tasa de evicción. Utilizado para el análisis posterior del rendimiento del sistema bajo distintas configuraciones.

El sistema no realiza consultas a una base de datos en tiempo de ejecución. Toda la información es precargada en memoria al iniciar el sistema.

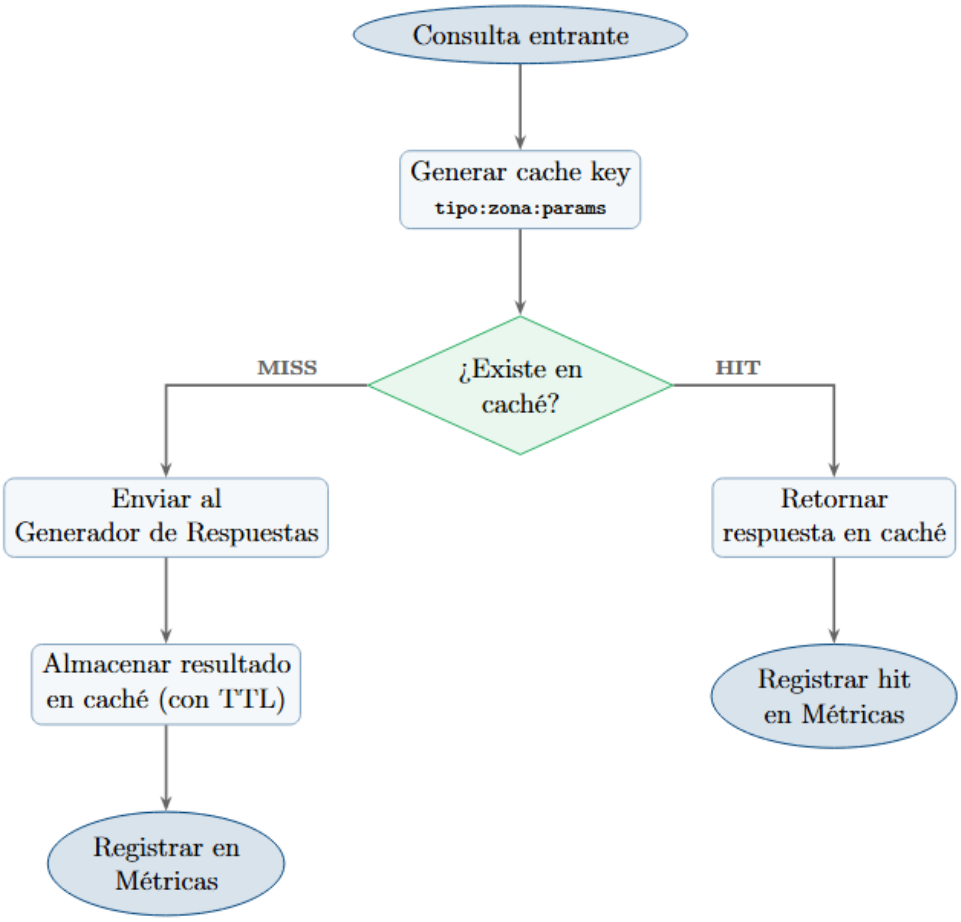

Figura 1: Flujo de desición interno del sistema caché

3. Flujo del Sistema

El flujo comienza con el Generador de Tráfico, que envía consultas al sistema de caché. Si la respuesta existe, se retorna inmediatamente. En caso contrario, la consulta es procesada por el Generador de Respuestas, que calcula el resultado y lo retorna al caché para su almacenamiento. Paralelamente, se registran métricas del sistema.

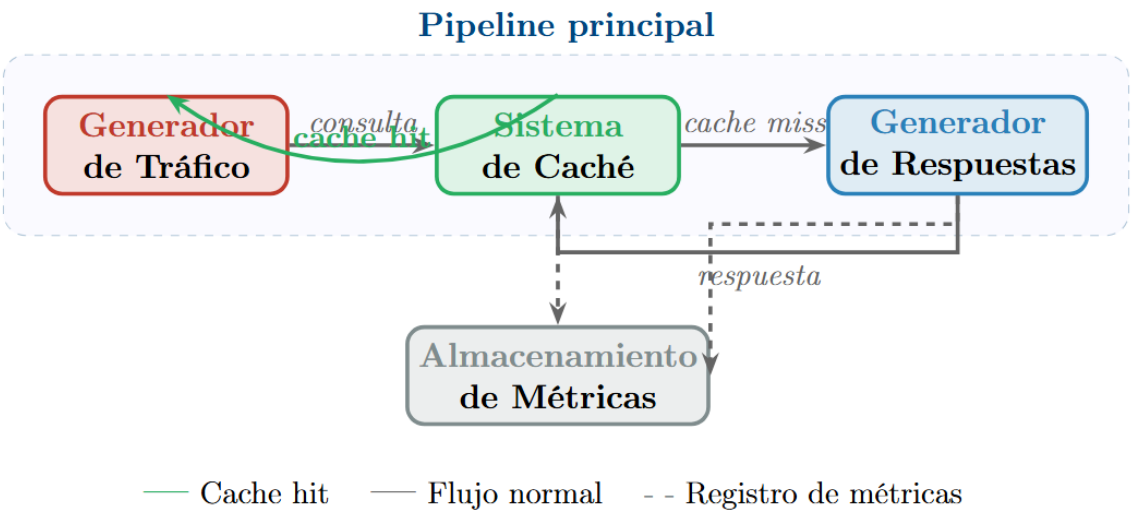

Figura 2: Arquitectura general del sistema

4. Datasets y Zonas de Consulta

4.1. Google Open Buildings — Región Metropolitana

El dataset provee, por edificación, los siguientes campos que son precargados en memoria por el Generador de Respuestas al iniciar el sistema:

Campo	Tipo	Descripción
`latitude`	`FLOAT`	Latitud del centroide
`longitude`	`FLOAT`	Longitud del centroide
`area_in_meters`	`FLOAT`	Área aproximada (m^2)
`confidence`	`FLOAT`	Confianza de detección [0,1]

Al arrancar, el sistema carga el subconjunto del dataset correspondiente a las cinco zonas de Santiago en una estructura en memoria (p.ej. un DataFrame). Las consultas Q1–Q5 operan directamente sobre esta estructura, sin necesidad de una base de datos externa.

4.2. Zonas Geográficas Predefinidas

Se definen cinco zonas fijas de Santiago como *bounding boxes*. Esto permite que el Generador de Tráfico reproduzca patrones de repetición realistas, favoreciendo los cache hits.

Zona (ID)	lat_min	lat_max	lon_min	lon_max
Providencia (Z1)	−33,445	−33,420	−70,640	−70,600
Las Condes (Z2)	−33,420	−33,390	−70,600	−70,550
Maipú (Z3)	−33,530	−33,490	−70,790	−70,740
Santiago Centro(Z4)	−33,460	−33,430	−70,670	−70,630
Pudahuel (Z5)	−33,470	−33,430	−70,810	−70,760

5. Tipos de Consultas

5.1. Q1 — Conteo de edificios en una zona

Descripción: Cuenta el número total de edificaciones detectadas dentro de una bounding box. Es la consulta más frecuente para estimar la densidad de puntos de entrega potenciales en un sector.

Parámetros: `bbox` (lat_min, lat_max, lon_min, lon_max), `confidence_min` (opcional, por defecto 0.0)

Cache key: `count:{zona_id}:conf={confidence_min}`

```python
def q1_count(zone_id, confidence_min=0.0):
    records = data[zone_id]  # registros precargados para la zona
    return sum(1 for r in records if r.confidence >= confidence_min)
```

Listing 1: Consulta Q1 — Conteo por zona (en memoria)

5.2. Q2 — Área promedio y área total de edificaciones

Descripción: Calcula el área promedio y el área total construida en una zona. Útil para clasificar sectores según tipo de construcción (residencial vs. comercial).

Parámetros: `bbox`, `confidence_min`

Cache key: `area:{zona_id}:conf={confidence_min}`

```python
def q2_area(zone_id, confidence_min=0.0):
    areas = [r.area for r in data[zone_id] if r.confidence >= confidence_min]
    return {"avg_area": mean(areas), "total_area": sum(areas), "n": len(areas)}
```

Listing 2: Consulta Q2 — Área por zona (en memoria)

5.3. Q3 — Densidad de edificaciones por km²

Descripción: Calcula la densidad de edificaciones por kilómetro cuadrado en una zona, normalizando por el área de la bounding box. Permite comparar sectores de distintos tamaños.

Parámetros: `bbox`, `confidence_min`

Cache key: `density:{zona_id}:conf={confidence_min}`

```
def q3_density(zone_id, confidence_min=0.0):
    count = q1_count(zone_id, confidence_min)
    area_km2 = zone_area_km2[zone_id]   #  rea  precalculada de la bbox
    return count / area_km2
```

Listing 3: Consulta Q3 — Densidad por zona (en memoria)

5.4. Q4 — Comparación de densidad entre dos zonas

Descripción: Compara la densidad de edificaciones entre dos zonas predefinidas. Las empresas de reparto utilizan esta consulta para priorizar sectores de expansión.

Parámetros: `zona_id_a`, `zona_id_b`, `confidence_min`

Cache key: `compare:density:{zona_a}:{zona_b}:conf={confidence_min}`

```
def q4_compare(zone_a, zone_b, confidence_min=0.0):
    da = q3_density(zone_a, confidence_min)
    db = q3_density(zone_b, confidence_min)
    return {"zone_a": da, "zone_b": db, "winner": zone_a if da > db else zone_b}
```

Listing 4: Consulta Q4 — Comparación entre zonas (en memoria)

5.5. Q5 — Distribución de confianza en una zona

Descripción: Calcula la distribución del score de confianza de detección en una zona, agrupado en intervalos. Permite evaluar la calidad del dato geoespacial antes de tomar decisiones operativas.

Parámetros: `bbox`, número de intervalos `bins` (por defecto 5)

Cache key: `confidence_dist:{zona_id}:bins={bins}`

```
def q5_confidence_dist(zone_id, bins=5):
    scores = [r.confidence for r in data[zone_id]]
    counts, edges = histogram(scores, bins=bins, range=(0, 1))
    return [{"bucket": i, "min": edges[i], "max": edges[i+1],
             "count": counts[i]} for i in range(bins)]
```

Listing 5: Consulta Q5 — Distribución de confianza (en memoria)

Análisis y Discusión

En esta sección, se debe realizar un análisis crítico y una discusión fundamentada sobre las decisiones de diseño y los resultados obtenidos. El análisis debe estar respaldado por datos empíricos, gráficos y tablas comparativas. A continuación, se detallan los puntos que deben abordar obligatoriamente:

Análisis del Comportamiento de la Caché

Impacto de la Distribución de Tráfico

Deben analizar y comparar el rendimiento del sistema de caché bajo las distintas distribuciones de tráfico que implementaron. Es necesario que respondan preguntas como:

- ¿Cómo varía la tasa de aciertos (*hit rate*) y de fallos (*miss rate*) entre una distribución y otra?
- ¿Qué distribución de tráfico genera un mayor estrés o beneficio para la caché y por qué?
- ¿Qué implicaciones tienen estos resultados para un escenario de aplicación real?

Efecto de los Parámetros: Tamaño y Política de Remoción

Deben presentar un análisis experimental sobre la configuración de la caché. Para ello, tienen que:

- Evaluar y comparar el rendimiento de, al menos, dos políticas de remoción (ej. LRU, LFU, FIFO).
- Analizar cómo el tamaño de la caché afecta su rendimiento. Evaluar impacto del tamaño en hit rate con 50 MB, 200 MB, 500 MB.
- Analizar cómo afecta el TTL al sistema caché para cada una de las consultas.

Tódo el análisis debe ser basado en las siguientes métricas:

Métrica	**Definición**
Hit rate	$\frac{hits}{hits+misses}$
Throughput	Consultas exitosas / segundo
Latencia p50/p95	Percentiles de tiempo de respuesta
Eviction rate	Evictions / minuto
Cache efficiency	$\frac{hits \cdot t_{cache} - misses \cdot t_{db}}{total}$

Requisitos de la Entrega

La evaluación de este hito se basará en los siguientes entregables:

- **Informe Técnico:** Un documento en formato LaTeX (entregado como PDF) que describa de manera concisa y rigurosa la arquitectura y el enfoque del sistema. El informe debe centrarse en el **análisis crítico** que justifique las decisiones tomadas y los resultados obtenidos. El contenido debe incluir:
  - Una descripción detallada de los componentes y las funcionalidades implementadas en cada módulo del sistema.
  - Una justificación fundamentada de las decisiones de diseño, así como de las tecnologías y metodologías empleadas.
  - Un análisis exhaustivo de los resultados, respaldado por datos empíricos. Se deben utilizar gráficos, tablas comparativas y otros recursos visuales para facilitar la interpretación y evaluación de las métricas definidas.
- **Video de Demostración:** Un video con una duración de 10 minutos donde se muestre el sistema en funcionamiento, explicando sus componentes y el flujo de datos.
- **Código Fuente:** El código fuente completo del proyecto, el cual deberá estar alojado en un repositorio público en GitHub o GitLab. El enlace al repositorio deberá incluirse tanto en la sección de comentarios de la plataforma CANVAS como en el informe técnico.
- **Archivos de Despliegue:** Un archivo `Dockerfile` y/o `docker-compose.yml` que permita la construcción y ejecución de todos los servicios implementados. El archivo `README.md` del repositorio deberá contener instrucciones claras y precisas para el despliegue y la ejecución del sistema.

Reglas y consideraciones de la entrega

- **Fecha de Entrega:** La entrega se realiza el día Viernes 17 de abril vía Canvas del curso.
- **Integrantes:** La tarea debe ser realizada en grupos de hasta **2** alumnos/as.
- **Tecnología:** se requiere Docker. El stack interno queda a elección del equipo, siempre que cada decisión esté justificada.
- **Ética y Autoría:** Se debe respetar el reglamento de la universidad en cuanto a plagio y autoría. Cualquier evidencia de copia o falta de autoría conllevará sanciones según lo establecido.